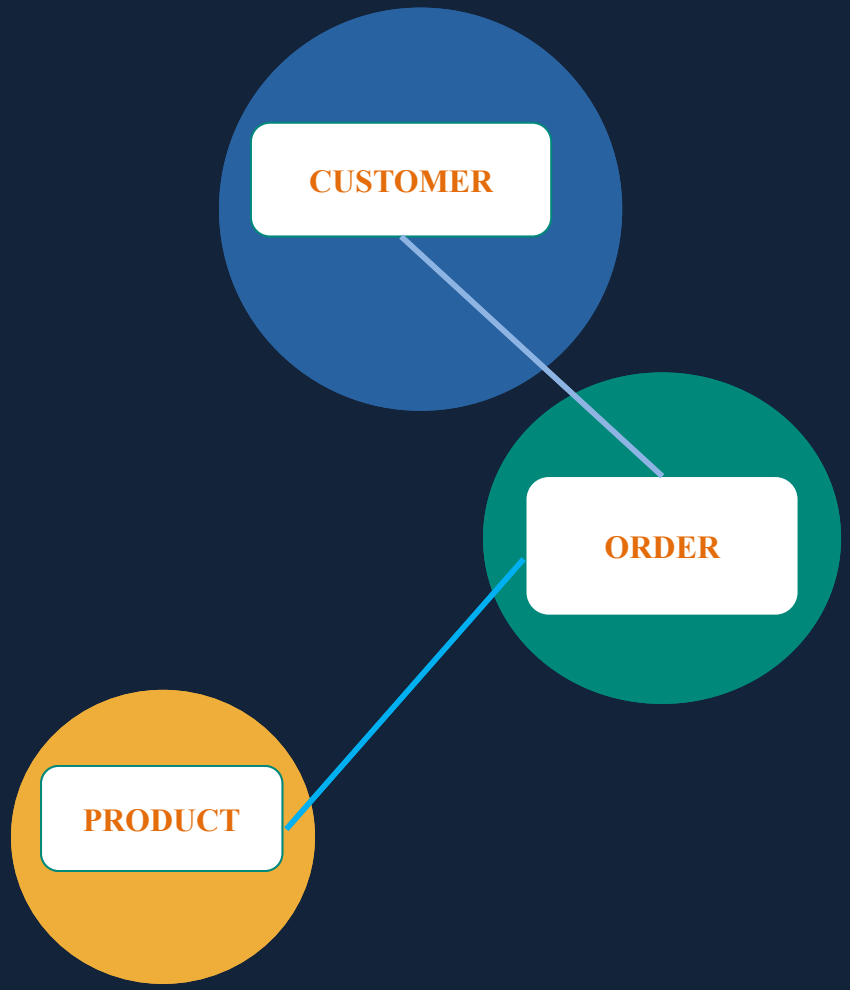
DATABASE SYSTEMS • SESSION 04

ER Modeling & Relational Design

Deep Review • Design Reasoning • Workshop



From drawing a model to defending a design decision

Warm-up: answer before we discuss

Think individually for 60 seconds; then compare with a partner.

A Can a table have more than one candidate key?

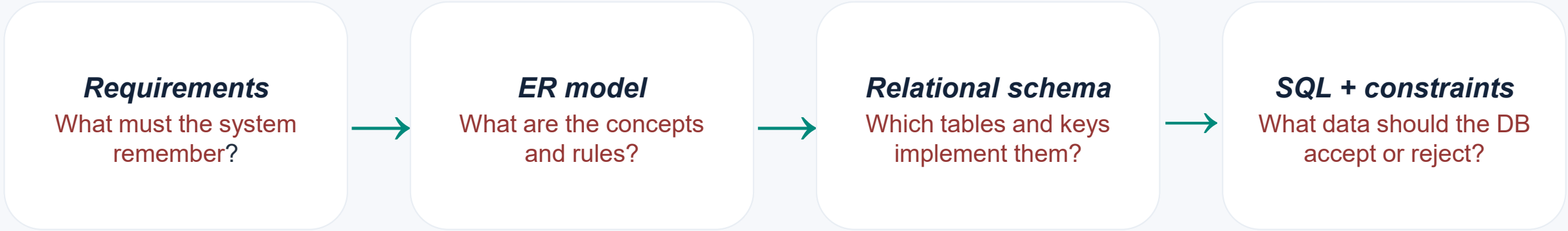
B Can a foreign key contain NULL?

C Does every M:N relationship mean we forgot an entity?

D Should every 1:1 relationship be merged?

Ask what the requirements allow—do not guess a universal rule.

The design pipeline: each stage answers a different question



A correct ER diagram can still become a poor schema if mapping decisions are careless.

Entity type, entity instance, and attribute

Entity type

CUSTOMER

A category of objects the system tracks.

Entity instance

Customer **#1042**

One particular customer.

Attribute

Email = sara@usc.ac.ir

A property of that customer.

Not every noun must become an entity. Ask about identity, properties, relationships, and lifecycle.

Should this concept become an entity?

Likely a separate entity when…

- It has several attributes of its own
- It participates in relationships
- It has an identity that must be referenced
- It changes independently or has its own lifecycle

May remain an attribute when…

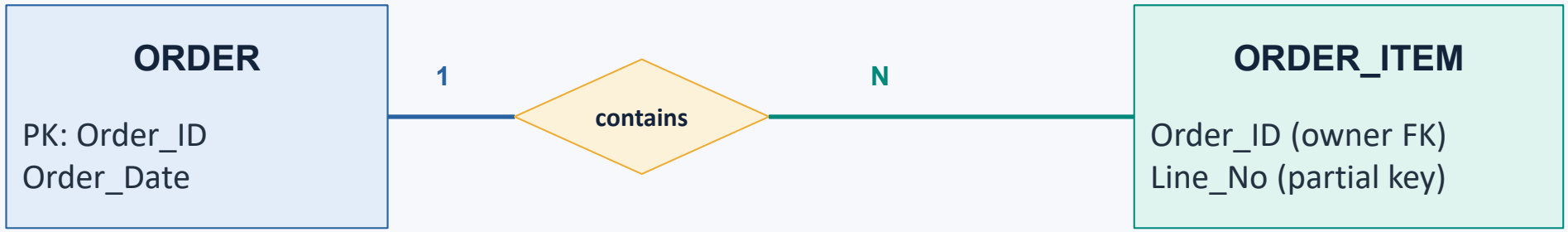
- It is a simple property of another entity
- It has no independent identity in this system
- No separate facts or relationships must be stored
- A separate entity adds no useful meaning

A shipping address may be an attribute or a reusable record, depending on requirements.

Strong vs. weak entity: identity depends on the model

ORDER — strong entity

ORDER_ITEM — dependent identity



Order_ID identifies an order on its own.

Often identified by (Order_ID, Line_No).

“Weak” concerns identification/dependence—not importance or number of attributes.

A dependent entity can be a regular table with a composite primary key.

ER notation: recognize the meaning of each symbol



Rectangle



Double rectangle



Diamond



Double diamond

Attribute notation

Oval; key attributes are underlined in classic Chen notation.

Participation / cardinality

Lines, double lines, 1/N or min–max labels express constraints.

Notation varies across textbooks and tools. Add a legend when needed.

Cardinality and participation are different questions

Cardinality

How many related instances? 1:1 • 1:N • M:N

Participation

Must an instance participate? Mandatory vs. optional

Example: a *CUSTOMER* may place zero or many *ORDERS*.



Every ORDER may require a CUSTOMER, while a CUSTOMER may have no orders.

Quick check: read the business rule carefully

A hospital appointment must be for exactly one patient. A patient may have many appointments—or none.

Relationship
PATIENT — books — APPOINTMENT

Cardinality
PATIENT 1 : N APPOINTMENT

Participation
Appointment must reference a patient

Can an appointment be booked before a patient record exists?

The answer depends on workflow, not on a universal ER rule.

What makes an attribute set a key?

A key distinguishes one row from every other row in the relation.

Uniqueness

No two rows share the key value or combination.

Minimality

A candidate key has no smaller uniquely identifying subset.

Good PK choice

Prefer stable, appropriate identifiers; uniqueness alone is not enough.

A relation may have several candidate keys. One is selected as the primary key; the others are alternate keys.

Key vocabulary: related terms, different meanings

Superkey	Any attribute set that uniquely identifies a row; may include extras.
Candidate key	A minimal superkey.
Primary key (PK)	The candidate key chosen as the main identifier.
Alternate key	A candidate key not selected as the PK.
Foreign key (FK)	Attribute(s) referencing a key in another or the same relation.
Composite key	A key made from two or more attributes.

A composite key may also be a candidate key, primary key, or foreign key.

Why can an enrollment key need three columns?

Business rule: a student may take the same course in different semesters.

Student_ID	Course_ID	Semester	Grade	Key?
S01	DB101	Fall-26	8	Yes (part)
S01	DB101	Spring-27	14	Yes (part)
S01	AI201	Fall-26	17	Yes (part)
S02	DB101	Fall-26	16	Yes (part)

No single column uniquely identifies every enrollment row.

If one enrollment per student-course-semester: PK = (Student_ID, Course_ID, Semester).

If repeated attempts are allowed, add Attempt_No or use Enrollment_ID.

Natural key vs. surrogate key

Natural key

Meaningful domain value.
Example: a unique identifier, if guaranteed and appropriate.
Risks: may change, be sensitive, or have exceptions.

Surrogate key

Artificial identifier such as Customer_ID.
Often stable and independent of business meaning.
Still enforce real-world uniqueness where required.

A surrogate PK does not remove the need for UNIQUE constraints on business identifiers.

Email should be UNIQUE only if the business rule guarantees one email per customer.

Key design challenge: library loans

A library can lend the same physical copy to the same member multiple times.

Option A

PK = (Member_ID, Copy_ID)

Cannot represent repeated loans.

Option B

PK = (Member_ID, Copy_ID, Loan_Date)

Could two loans occur on the same date?

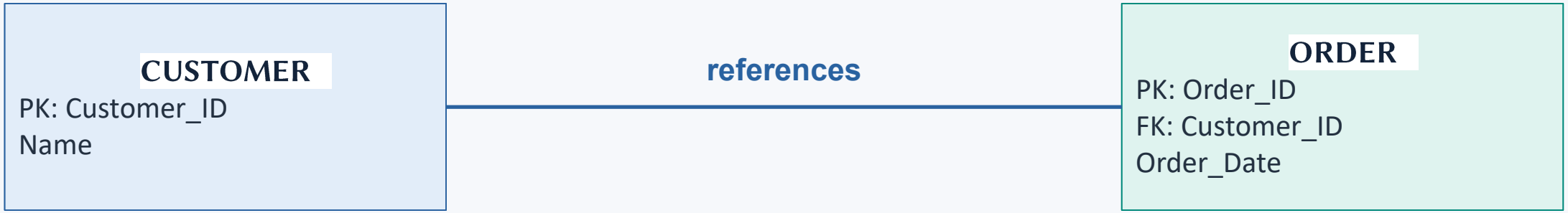
Option C

Loan_ID as PK

Member_ID and Copy_ID as FKs; store date/time.

Discuss: which design best captures the rules? What if a loan can be renewed?

Why do we need a foreign key?



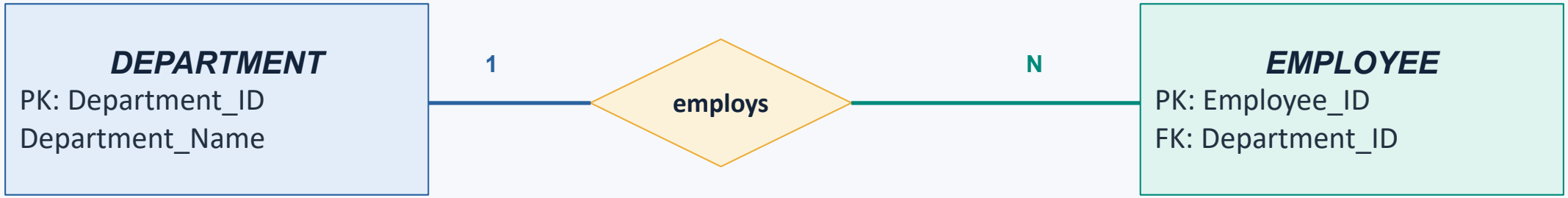
Without an FK, an order might reference Customer_ID = 9999 even if no such customer exists.

With an FK, the database checks the referenced customer exists (unless NULL is allowed).

Referential integrity

The relationship becomes an enforceable rule—not merely a line in a diagram.

Mapping 1:N: put the FK on the many side

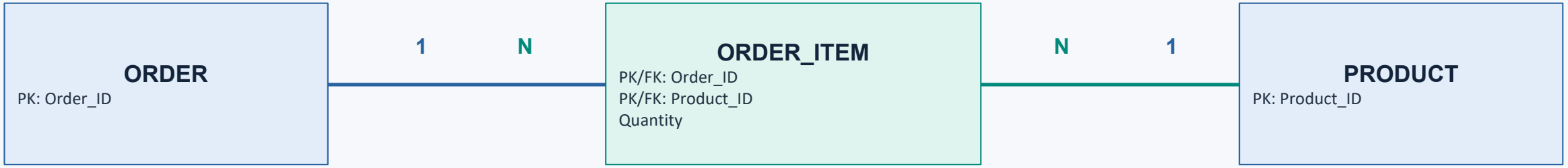


Each employee references at most one department; a department can have many employees.

Mandatory assignment
Department_ID is NOT NULL.

Optional assignment
Department_ID may be NULL if the rule permits.

Mapping M:N: use an associative table



An order can contain many products; a product can appear in many orders.

Why ORDER_ITEM?

Resolves M:N into two 1:N relationships; stores Quantity per pair.

Possible key

(Order_ID, Product_ID), if each product appears once per order.

If a product may appear on multiple lines, include Line_No in the key.

Does every M:N mean we missed an entity?

Not necessarily

An M:N can be a real business relationship. Relational implementation still needs a junction table.

Often the table has meaning

If the association has attributes (Quantity, Grade, Role, Start_Date) or a lifecycle, an associative entity is natural.

Example: STUDENT — enrolls in — COURSE; enrollment has Semester, Grade, and Status.

Ask: “What facts belong to the association itself?” rather than “Did I forget an entity?”

Should a 1:1 relationship always be merged?

Merging may be reasonable when…

- Same lifecycle
- Usually present together
- Similar access/security needs
- Separation adds no useful meaning

Keeping separate may be better when…

- The second part is optional
- It has a distinct lifecycle
- Different access/security is needed
- It is accessed separately

1:1 does not automatically mean “merge,” and 1:N is not inherently more desirable.

EMPLOYEE and PAYROLL_PROFILE may be separate for access control; enforce 1:1 with a UNIQUE FK.

Common design mistakes to catch early

01
Treating every noun as an entity

02
Choosing a key before checking business rules

03
Using a composite key without explaining uniqueness

04
Assuming an FK must always be non-NULL

05
Turning M:N into one FK on one side

06
Assuming 1:1 tables must always merge

Fix the assumption first; then fix the diagram or schema.

Design review checklist

- ✓ Can I state the business rule in one clear sentence?
- ✓ Does every entity have a defensible identifier?
- ✓ Have I separated cardinality from optionality?
- ✓ Are M:N relationships mapped through an associative table?
- ✓ Are FKs on the correct side and nullable only when allowed?
- ✓ Have I documented assumptions and exceptions?

A design is not finished until others can understand and challenge its assumptions.

Workshop: model a small online store

REQUIREMENTS

1. Each customer has a unique Customer_ID and may place zero or many orders.
2. Each order belongs to exactly one customer and has an Order_Date.
3. Each product has a unique Product_ID and a current Price.
4. An order contains one or more products; each product may appear in many orders.
5. Quantity ordered belongs to the order-product association.

Task: list entities, attributes, PKs, FKs, cardinalities, and one assumption.

Pairs: 6 minutes • Compare: 4 minutes • Defend: 2 minutes

Defend your design: explain the trade-offs

Q1 Why is ORDER_ITEM a table rather than just an attribute?

Q2 Why is Product_ID not enough to identify an ORDER_ITEM row?

Q3 What rule would make Customer_ID in ORDER NOT NULL?

Q4 What changes if a product may appear twice as separate lines?

A strong answer includes both the decision and the business rule that justifies it.

From design decisions to database constraints

Primary key
Uniquely identifies each row; generally NOT NULL and UNIQUE.

Foreign key
Referenced value must exist—or be NULL if permitted.

UNIQUE / NOT NULL
Enforce alternate uniqueness and mandatory values.

The ER model communicates intent; SQL constraints make important rules enforceable.

Next: PK, FK, candidate/alternate keys, NULL behavior, referential actions, and invalid INSERT tests.

What should the database reject?

Duplicate Customer_ID	Reject: primary-key violation
ORDER references unknown Customer_ID	Reject: foreign-key violation
Required Order_Date is NULL	Reject if NOT NULL is specified
Same customer email appears twice	Reject only if UNIQUE is part of the design

The database rejects only rules that have actually been defined and enforced.

Exit ticket: 3-minute individual check

1. In one sentence, explain why a foreign key is needed.

2. Give one reason a composite key may be necessary.

3. Correct this claim: “Every 1:1 relationship should be merged.”

4. What information does ORDER_ITEM store that ORDER and PRODUCT cannot store alone?

Write independently; use business rules, not memorized slogans.